```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(r"C:\Users\hamsa\Desktop\Financial_loan_dataset.csv")

df.head()
```

```
   ISSUE_DATE       ID PURPOSE VERIFICATION_STATUS GRADE
HOME_OWNERSHIP  \
0  01-01-2021  1072053     car     Source Verified     E
RENT
1  01-01-2021  1068350     car            Verified     A
MORTGAGE
2  05-01-2021  1069243     car        Not Verified     C
RENT
3  08-01-2021   207910     car        Not Verified     A
MORTGAGE
4  08-01-2021   211723     car        Not Verified     C
OWN

  ADDRESS_STATE  LOAN_STATUS EMP_LENGTH              EMP_TITLE  ...
MEMBER_ID  \
0            CA   Fully Paid    9 years         MKC Accounting  ...
1288686
1            IL   Fully Paid  10+ years          J&J Steel Inc  ...
1302971
2            CA  Charged Off    4 years  Chemat Technology Inc  ...
1304116
3            FL  Charged Off   < 1 year                    NaN  ...
183496
4            MD  Charged Off    4 years          self employed  ...
211606

  SUB_GRADE         TERM  ANNUAL_INCOME     DTI INSTALLMENT
INT_RATE  \
0        E1   36 months        48000.0  0.0535      109.43    0.1864

1        A1   36 months        83000.0  0.0231      106.53    0.0603

2        C5   36 months        50000.0  0.2088      421.65    0.1596

3        A2   36 months       120000.0  0.0767       69.14    0.0743

4        C2   36 months        72000.0  0.0512      262.26    0.1109


   LOAN_AMOUNT  TOTAL_ACC  TOTAL_PAYMENT
0         3000          4           3939
1         3500         28           3835
```

```
2        12000         11           3522
3         2225         20           2508
4         8000          8           2135

[5 rows x 23 columns]
```

```
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (38576, 29)
```

```
print("\nColumn Names:")
print(df.columns.tolist())
```

```
Column Names:
['ISSUE_DATE', 'ID', 'PURPOSE', 'VERIFICATION_STATUS', 'GRADE',
'HOME_OWNERSHIP', 'ADDRESS_STATE', 'LOAN_STATUS', 'EMP_LENGTH',
'EMP_TITLE', 'LAST_CREDIT_PULL_DATE', 'LAST_PAYMENT_DATE',
'NEXT_PAYMENT_DATE', 'MEMBER_ID', 'SUB_GRADE', 'TERM',
'ANNUAL_INCOME', 'DTI', 'INSTALLMENT', 'INT_RATE', 'LOAN_AMOUNT',
'TOTAL_ACC', 'TOTAL_PAYMENT']
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38576 entries, 0 to 38575
Data columns (total 23 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   ISSUE_DATE              38576 non-null  object
 1   ID                      38576 non-null  int64
 2   PURPOSE                 38576 non-null  object
 3   VERIFICATION_STATUS     38576 non-null  object
 4   GRADE                   38576 non-null  object
 5   HOME_OWNERSHIP          38576 non-null  object
 6   ADDRESS_STATE           38576 non-null  object
 7   LOAN_STATUS             38576 non-null  object
 8   EMP_LENGTH              38576 non-null  object
 9   EMP_TITLE               37138 non-null  object
 10  LAST_CREDIT_PULL_DATE   38576 non-null  object
 11  LAST_PAYMENT_DATE       38576 non-null  object
 12  NEXT_PAYMENT_DATE       38576 non-null  object
 13  MEMBER_ID               38576 non-null  int64
 14  SUB_GRADE               38576 non-null  object
 15  TERM                    38576 non-null  object
 16  ANNUAL_INCOME           38576 non-null  float64
 17  DTI                     38576 non-null  float64
 18  INSTALLMENT             38576 non-null  float64
```

```
 19  INT_RATE                38576 non-null  float64
 20  LOAN_AMOUNT             38576 non-null  int64
 21  TOTAL_ACC               38576 non-null  int64
 22  TOTAL_PAYMENT           38576 non-null  int64
dtypes: float64(4), int64(5), object(14)
memory usage: 6.8+ MB
```

```
df.describe()
```

```
                  ID     MEMBER_ID  ANNUAL_INCOME           DTI
INSTALLMENT  \
count  3.857600e+04  3.857600e+04   3.857600e+04  38576.000000
38576.000000
mean   6.810371e+05  8.476515e+05   6.964454e+04      0.133274
326.862965
std    2.113246e+05  2.668105e+05   6.429368e+04      0.066662
209.092000
min    5.473400e+04  7.069900e+04   4.000000e+03      0.000000
15.690000
25%    5.135170e+05  6.629788e+05   4.150000e+04      0.082100
168.450000
50%    6.627280e+05  8.473565e+05   6.000000e+04      0.134200
283.045000
75%    8.365060e+05  1.045652e+06   8.320050e+04      0.185900
434.442500
max    1.077501e+06  1.314167e+06   6.000000e+06      0.299900
1305.190000
```

```
           INT_RATE   LOAN_AMOUNT     TOTAL_ACC  TOTAL_PAYMENT
count  38576.000000  38576.000000  38576.000000   38576.000000
mean       0.120488  11296.066855     22.132544   12263.348533
std        0.037164   7460.746022     11.392282    9051.104777
min        0.054200    500.000000      2.000000      34.000000
25%        0.093200   5500.000000     14.000000    5633.000000
50%        0.118600  10000.000000     20.000000   10042.000000
75%        0.145900  15000.000000     29.000000   16658.000000
max        0.245900  35000.000000     90.000000   58564.000000
```

```
df.describe(include='object')
```

```
        ISSUE_DATE             PURPOSE VERIFICATION_STATUS  GRADE  \
count        38576               38576                38576  38576
unique          65                  14                    3      7
top     11-12-2021  Debt consolidation         Not Verified      B
freq          2188               18214                16464  11674

       HOME_OWNERSHIP ADDRESS_STATE LOAN_STATUS EMP_LENGTH
EMP_TITLE  \
count           38576         38576       38576      38576     37138
```

```
unique                5             50            3          11      28525

top                RENT             CA   Fully Paid   10+ years    US Army

freq              18439           6894        32145        8870        135


       LAST_CREDIT_PULL_DATE LAST_PAYMENT_DATE NEXT_PAYMENT_DATE
SUB_GRADE  \
count                  38576             38576              38576
38576
unique                   107               102                102
35
top                16-05-2021        16-05-2021        16-06-2021
B3
freq                   10041              1209               1209
2834

               TERM
count         38576
unique            2
top       36 months
freq          28237
```

```
missing_values = df.isnull().sum()

missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_data = pd.DataFrame({
    'Missing Values': missing_values,
    'Missing Percentage': missing_percentage
})

missing_data
```

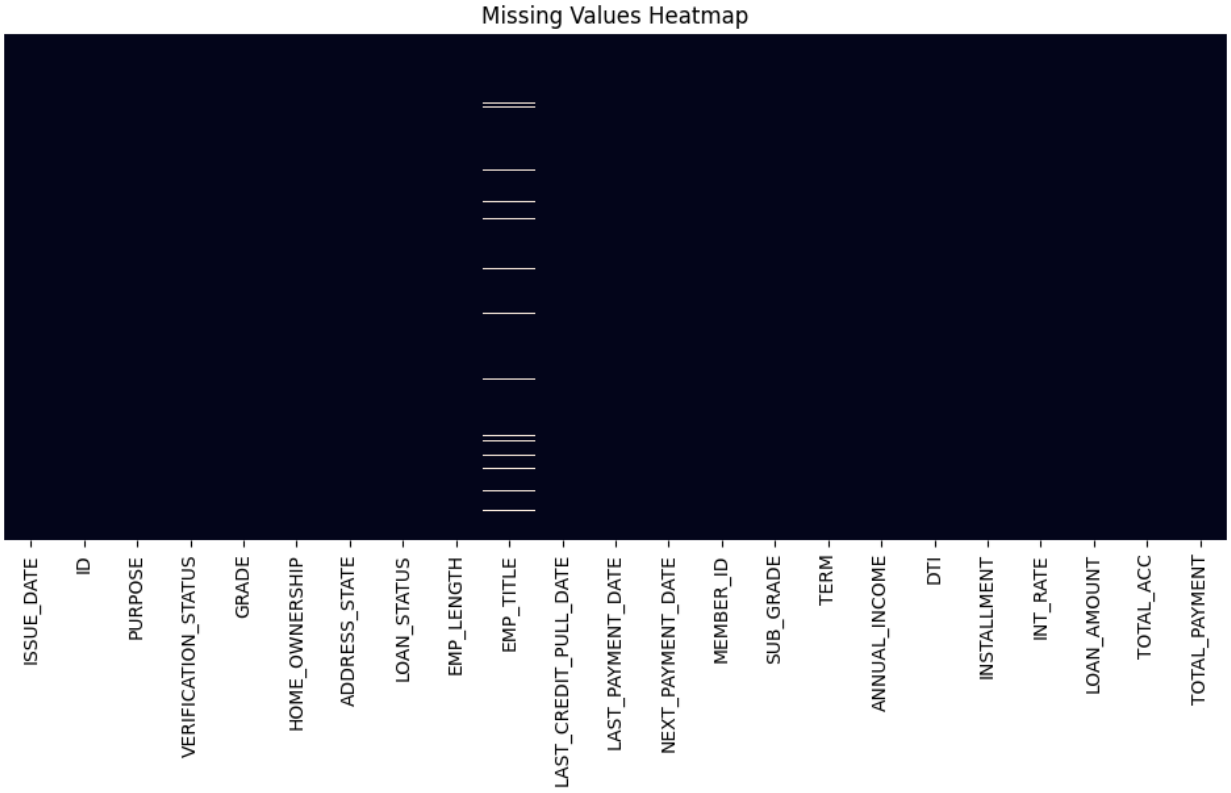
```
                      Missing Values  Missing Percentage
ISSUE_DATE                         0            0.000000
ID                                 0            0.000000
PURPOSE                            0            0.000000
VERIFICATION_STATUS                0            0.000000
GRADE                              0            0.000000
HOME_OWNERSHIP                     0            0.000000
ADDRESS_STATE                      0            0.000000
LOAN_STATUS                        0            0.000000
EMP_LENGTH                         0            0.000000
EMP_TITLE                       1438            3.727706
LAST_CREDIT_PULL_DATE              0            0.000000
LAST_PAYMENT_DATE                  0            0.000000
NEXT_PAYMENT_DATE                  0            0.000000
MEMBER_ID                          0            0.000000
```

```
SUB_GRADE                       0             0.000000
TERM                            0             0.000000
ANNUAL_INCOME                   0             0.000000
DTI                             0             0.000000
INSTALLMENT                     0             0.000000
INT_RATE                        0             0.000000
LOAN_AMOUNT                     0             0.000000
TOTAL_ACC                       0             0.000000
TOTAL_PAYMENT                   0             0.000000
```

```
plt.figure(figsize=(12, 5))

sns.heatmap(df.isnull(), cbar=False, yticklabels=False)

plt.title('Missing Values Heatmap')
plt.show()
```




```
print("Total Duplicate Rows:", df.duplicated().sum())

print("Duplicate Loan IDs:", df['ID'].duplicated().sum())
```

```
Total Duplicate Rows: 0
Duplicate Loan IDs: 0
```

```
df.drop_duplicates(inplace=True)

print("Dataset Shape After Removing Duplicates:", df.shape)
```

```
Dataset Shape After Removing Duplicates: (38576, 23)
```

```
df.isnull().sum()
```

```
ISSUE_DATE                  0
ID                          0
PURPOSE                     0
VERIFICATION_STATUS         0
GRADE                       0
HOME_OWNERSHIP              0
ADDRESS_STATE               0
LOAN_STATUS                 0
EMP_LENGTH                  0
EMP_TITLE                1438
LAST_CREDIT_PULL_DATE       0
LAST_PAYMENT_DATE           0
NEXT_PAYMENT_DATE           0
MEMBER_ID                   0
SUB_GRADE                   0
TERM                        0
ANNUAL_INCOME               0
DTI                         0
INSTALLMENT                 0
INT_RATE                    0
LOAN_AMOUNT                 0
TOTAL_ACC                   0
TOTAL_PAYMENT               0
dtype: int64
```

```
df['EMP_TITLE'] = df['EMP_TITLE'].fillna('Unknown')

df.isnull().sum()
```

```
ISSUE_DATE                0
ID                        0
PURPOSE                   0
VERIFICATION_STATUS       0
GRADE                     0
HOME_OWNERSHIP            0
ADDRESS_STATE             0
LOAN_STATUS               0
EMP_LENGTH                0
EMP_TITLE                 0
LAST_CREDIT_PULL_DATE     0
LAST_PAYMENT_DATE         0
NEXT_PAYMENT_DATE         0
MEMBER_ID                 0
```

```
SUB_GRADE                 0
TERM                      0
ANNUAL_INCOME             0
DTI                       0
INSTALLMENT               0
INT_RATE                  0
LOAN_AMOUNT               0
TOTAL_ACC                 0
TOTAL_PAYMENT             0
dtype: int64
```

```
date_columns = [
    'ISSUE_DATE',
    'LAST_CREDIT_PULL_DATE',
    'LAST_PAYMENT_DATE',
    'NEXT_PAYMENT_DATE'
]

for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors='coerce')

numeric_columns = [
    'ANNUAL_INCOME',
    'DTI',
    'INSTALLMENT',
    'INT_RATE',
    'LOAN_AMOUNT',
    'TOTAL_ACC',
    'TOTAL_PAYMENT'
]

for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38576 entries, 0 to 38575
Data columns (total 23 columns):
 #   Column                Non-Null Count  Dtype         
---  ------                --------------  -----         
 0   ISSUE_DATE            38572 non-null  datetime64[ns]
 1   ID                    38576 non-null  int64         
 2   PURPOSE               38576 non-null  object        
 3   VERIFICATION_STATUS   38576 non-null  object        
 4   GRADE                 38576 non-null  object        
 5   HOME_OWNERSHIP        38576 non-null  object        
 6   ADDRESS_STATE         38576 non-null  object        
 7   LOAN_STATUS           38576 non-null  object        
 8   EMP_LENGTH            38576 non-null  object
```

```
 9   EMP_TITLE               38576 non-null  object
 10  LAST_CREDIT_PULL_DATE   38576 non-null  datetime64[ns]
 11  LAST_PAYMENT_DATE       38576 non-null  datetime64[ns]
 12  NEXT_PAYMENT_DATE       38576 non-null  datetime64[ns]
 13  MEMBER_ID               38576 non-null  int64
 14  SUB_GRADE               38576 non-null  object
 15  TERM                    38576 non-null  object
 16  ANNUAL_INCOME           38576 non-null  float64
 17  DTI                     38576 non-null  float64
 18  INSTALLMENT             38576 non-null  float64
 19  INT_RATE                38576 non-null  float64
 20  LOAN_AMOUNT             38576 non-null  int64
 21  TOTAL_ACC               38576 non-null  int64
 22  TOTAL_PAYMENT           38576 non-null  int64
dtypes: datetime64[ns](4), float64(4), int64(5), object(10)
memory usage: 6.8+ MB

C:\Users\hamsa\AppData\Local\Temp\ipykernel_8664\2331547252.py:9:
UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the
default) was specified. Pass `dayfirst=True` or specify a format to
silence this warning.
  df[col] = pd.to_datetime(df[col], errors='coerce')
C:\Users\hamsa\AppData\Local\Temp\ipykernel_8664\2331547252.py:9:
UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the
default) was specified. Pass `dayfirst=True` or specify a format to
silence this warning.
  df[col] = pd.to_datetime(df[col], errors='coerce')
C:\Users\hamsa\AppData\Local\Temp\ipykernel_8664\2331547252.py:9:
UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the
default) was specified. Pass `dayfirst=True` or specify a format to
silence this warning.
  df[col] = pd.to_datetime(df[col], errors='coerce')
```

```
categorical_columns = [
    'PURPOSE',
    'VERIFICATION_STATUS',
    'GRADE',
    'HOME_OWNERSHIP',
    'ADDRESS_STATE',
    'LOAN_STATUS',
    'EMP_LENGTH',
    'SUB_GRADE',
    'TERM'
]

for col in categorical_columns:
    df[col] = df[col].astype('string').str.strip()

df.head()
```

```
  ISSUE_DATE       ID PURPOSE VERIFICATION_STATUS GRADE HOME_OWNERSHIP
\
0 2021-01-01  1072053     car     Source Verified     E           RENT

1 2021-01-01  1068350     car            Verified     A       MORTGAGE

2 2021-05-01  1069243     car        Not Verified     C           RENT

3 2021-08-01   207910     car        Not Verified     A       MORTGAGE

4 2021-08-01   211723     car        Not Verified     C            OWN


  ADDRESS_STATE  LOAN_STATUS EMP_LENGTH              EMP_TITLE  ...
MEMBER_ID  \
0            CA   Fully Paid    9 years         MKC Accounting  ...
1288686
1            IL   Fully Paid  10+ years          J&J Steel Inc  ...
1302971
2            CA  Charged Off    4 years  Chemat Technology Inc  ...
1304116
3            FL  Charged Off   < 1 year                Unknown  ...
183496
4            MD  Charged Off    4 years          self employed  ...
211606

  SUB_GRADE       TERM  ANNUAL_INCOME     DTI INSTALLMENT  INT_RATE  \
0        E1  36 months        48000.0  0.0535      109.43    0.1864
1        A1  36 months        83000.0  0.0231      106.53    0.0603
2        C5  36 months        50000.0  0.2088      421.65    0.1596
3        A2  36 months       120000.0  0.0767       69.14    0.0743
4        C2  36 months        72000.0  0.0512      262.26    0.1109

   LOAN_AMOUNT  TOTAL_ACC  TOTAL_PAYMENT
0         3000          4           3939
1         3500         28           3835
2        12000         11           3522
3         2225         20           2508
4         8000          8           2135

[5 rows x 23 columns]
```

```
for col in categorical_columns:
    print(f"\n{col}:")
    print(df[col].unique())
```

```
PURPOSE:
<StringArray>
[               'car',        'credit card', 'Debt consolidation',
       'educational',   'home improvement',     'major purchase',
```

```
            'medical',             'moving',              'other',
     'small business',           'vacation',            'wedding',
              'house',   'renewable_energy']
Length: 14, dtype: string

VERIFICATION_STATUS:
<StringArray>
['Source Verified', 'Verified', 'Not Verified']
Length: 3, dtype: string

GRADE:
<StringArray>
['E', 'A', 'C', 'B', 'D', 'F', 'G']
Length: 7, dtype: string

HOME_OWNERSHIP:
<StringArray>
['RENT', 'MORTGAGE', 'OWN', 'NONE', 'OTHER']
Length: 5, dtype: string

ADDRESS_STATE:
<StringArray>
['CA', 'IL', 'FL', 'MD', 'MN', 'MA', 'WA', 'CO', 'TX', 'GA', 'PA',
'NC', 'VA',
 'NJ', 'OH', 'NY', 'RI', 'CT', 'MI', 'DC', 'LA', 'IA', 'AZ', 'NM',
'AR', 'NV',
 'OR', 'WI', 'MO', 'VT', 'AL', 'SC', 'HI', 'MS', 'OK', 'KS', 'NH',
'KY', 'AK',
 'WV', 'UT', 'DE', 'MT', 'WY', 'SD', 'ID', 'TN', 'NE', 'IN', 'ME']
Length: 50, dtype: string

LOAN_STATUS:
<StringArray>
['Fully Paid', 'Charged Off', 'Current']
Length: 3, dtype: string

EMP_LENGTH:
<StringArray>
[  '9 years', '10+ years',   '4 years',  '< 1 year',   '3 years',   '2
years',
   '7 years',    '1 year',   '6 years',   '5 years',   '8 years']
Length: 11, dtype: string

SUB_GRADE:
<StringArray>
['E1', 'A1', 'C5', 'A2', 'C2', 'A4', 'B1', 'B4', 'D1', 'A3', 'A5',
'B2', 'C1',
 'D2', 'F2', 'E4', 'C3', 'C4', 'D4', 'B3', 'D3', 'F3', 'B5', 'D5',
'E5', 'E2',
 'G5', 'E3', 'F1', 'F5', 'G4', 'F4', 'G3', 'G1', 'G2']
```

```
Length: 35, dtype: string

TERM:
<StringArray>
['36 months', '60 months']
Length: 2, dtype: string
```

```python
# Extract year and month
df['ISSUE_YEAR'] = df['ISSUE_DATE'].dt.year

df['ISSUE_MONTH'] = df['ISSUE_DATE'].dt.month

df['ISSUE_MONTH_NAME'] = df['ISSUE_DATE'].dt.month_name()

# Convert interest rate and DTI into percentages
df['INTEREST_RATE_PCT'] = df['INT_RATE'] * 100

df['DTI_PCT'] = df['DTI'] * 100

# Classify loans
df['LOAN_CATEGORY'] = df['LOAN_STATUS'].map({
    'Fully Paid': 'Good Loan',
    'Current': 'Good Loan',
    'Charged Off': 'Bad Loan'
})

df.head()
```

```
  ISSUE_DATE       ID PURPOSE VERIFICATION_STATUS GRADE HOME_OWNERSHIP
\
0 2021-01-01  1072053     car     Source Verified     E           RENT

1 2021-01-01  1068350     car            Verified     A       MORTGAGE

2 2021-05-01  1069243     car        Not Verified     C           RENT

3 2021-08-01   207910     car        Not Verified     A       MORTGAGE

4 2021-08-01   211723     car        Not Verified     C            OWN


  ADDRESS_STATE  LOAN_STATUS EMP_LENGTH              EMP_TITLE  ...
INT_RATE  \
0            CA   Fully Paid    9 years         MKC Accounting  ...
0.1864
1            IL   Fully Paid  10+ years          J&J Steel Inc  ...
0.0603
2            CA  Charged Off    4 years  Chemat Technology Inc  ...
0.1596
3            FL  Charged Off   < 1 year                Unknown  ...
0.0743
```

```
4            MD  Charged Off    4 years          self employed  ...
0.1109

  LOAN_AMOUNT TOTAL_ACC  TOTAL_PAYMENT ISSUE_YEAR ISSUE_MONTH  \
0        3000         4           3939     2021.0         1.0
1        3500        28           3835     2021.0         1.0
2       12000        11           3522     2021.0         5.0
3        2225        20           2508     2021.0         8.0
4        8000         8           2135     2021.0         8.0

   ISSUE_MONTH_NAME  INTEREST_RATE_PCT  DTI_PCT  LOAN_CATEGORY
0           January              18.64     5.35      Good Loan
1           January               6.03     2.31      Good Loan
2               May              15.96    20.88       Bad Loan
3            August               7.43     7.67       Bad Loan
4            August              11.09     5.12       Bad Loan

[5 rows x 29 columns]
```

```
print("Final Dataset Shape:", df.shape)
```

```
Final Dataset Shape: (38576, 29)
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
ISSUE_DATE                4
ID                        0
PURPOSE                   0
VERIFICATION_STATUS       0
GRADE                     0
HOME_OWNERSHIP            0
ADDRESS_STATE             0
LOAN_STATUS               0
EMP_LENGTH                0
EMP_TITLE                 0
LAST_CREDIT_PULL_DATE     0
LAST_PAYMENT_DATE         0
NEXT_PAYMENT_DATE         0
MEMBER_ID                 0
SUB_GRADE                 0
TERM                      0
ANNUAL_INCOME             0
DTI                       0
INSTALLMENT               0
INT_RATE                  0
LOAN_AMOUNT               0
TOTAL_ACC                 0
TOTAL_PAYMENT             0
```

```
ISSUE_YEAR                4
ISSUE_MONTH               4
ISSUE_MONTH_NAME          4
INTEREST_RATE_PCT         0
DTI_PCT                   0
LOAN_CATEGORY             0
dtype: int64
```

```
print("\nDuplicate Rows:", df.duplicated().sum())
```

```
Duplicate Rows: 0
```

```
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
ISSUE_DATE                datetime64[ns]
ID                                 int64
PURPOSE                  string[python]
VERIFICATION_STATUS      string[python]
GRADE                    string[python]
HOME_OWNERSHIP           string[python]
ADDRESS_STATE            string[python]
LOAN_STATUS              string[python]
EMP_LENGTH               string[python]
EMP_TITLE                        object
LAST_CREDIT_PULL_DATE    datetime64[ns]
LAST_PAYMENT_DATE        datetime64[ns]
NEXT_PAYMENT_DATE        datetime64[ns]
MEMBER_ID                          int64
SUB_GRADE                string[python]
TERM                     string[python]
ANNUAL_INCOME                   float64
DTI                             float64
INSTALLMENT                     float64
INT_RATE                        float64
LOAN_AMOUNT                       int64
TOTAL_ACC                         int64
TOTAL_PAYMENT                     int64
ISSUE_YEAR                      float64
ISSUE_MONTH                     float64
ISSUE_MONTH_NAME                 object
INTEREST_RATE_PCT               float64
DTI_PCT                         float64
LOAN_CATEGORY                    object
dtype: object
```

```
#Categorical columns analysis
categorical_columns = [
```

```
    'LOAN_STATUS',
]

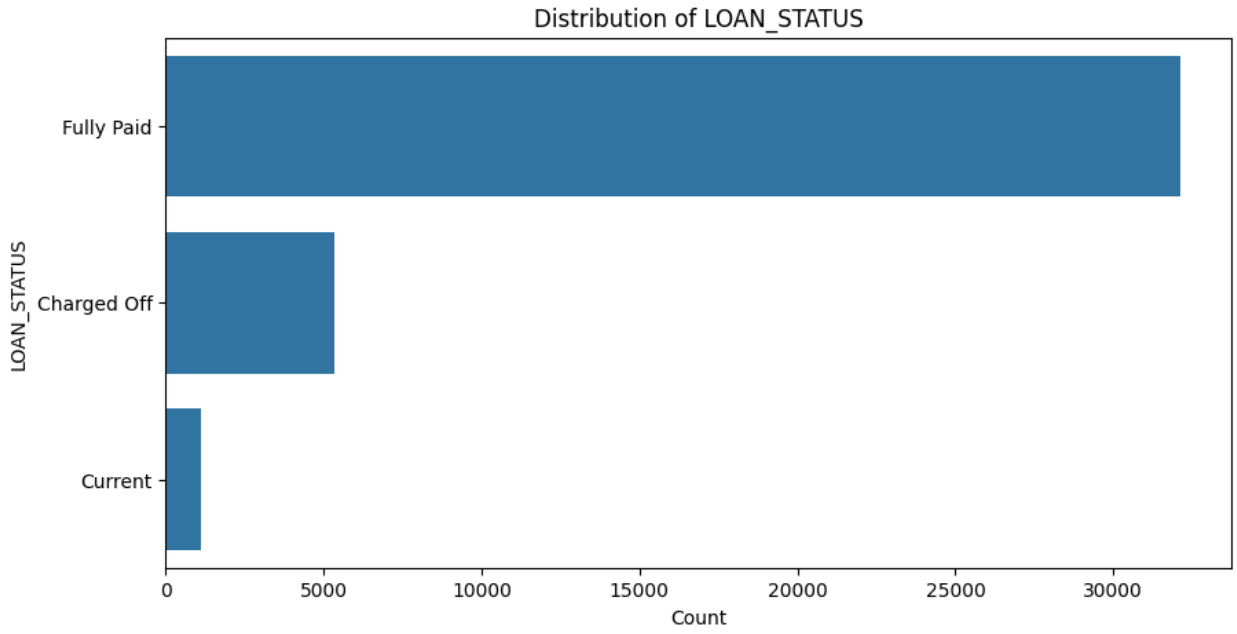
for col in categorical_columns:

    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```




```
categorical_columns = [
    'PURPOSE',
]

for col in categorical_columns:

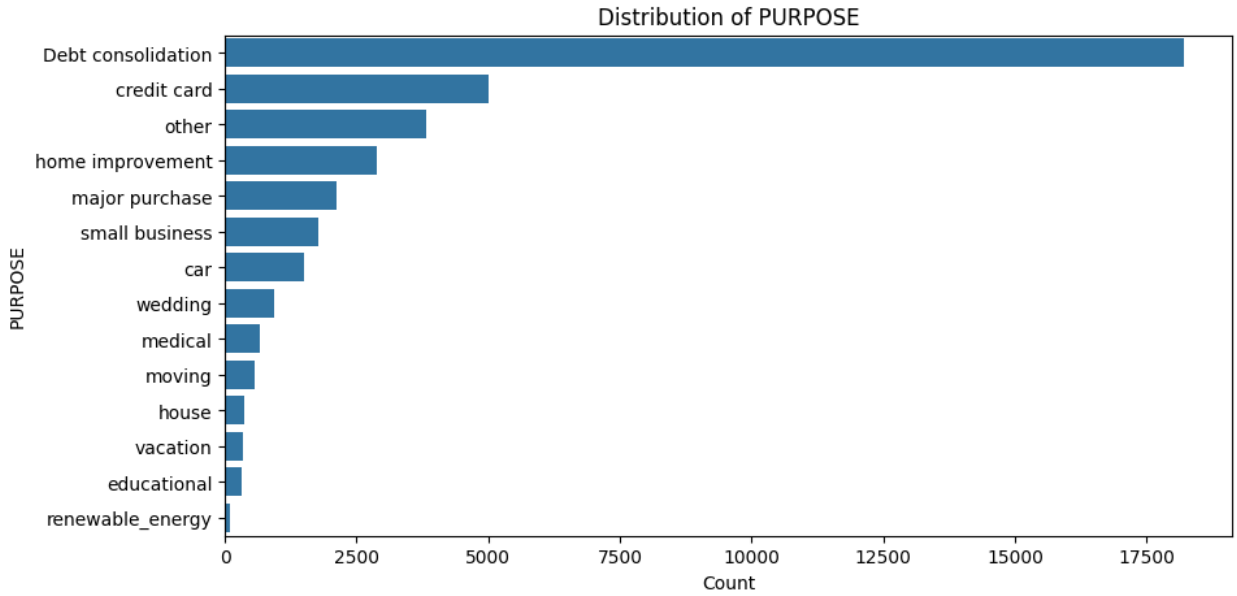
    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
```

```
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```




```
categorical_columns = [
    'GRADE',
]

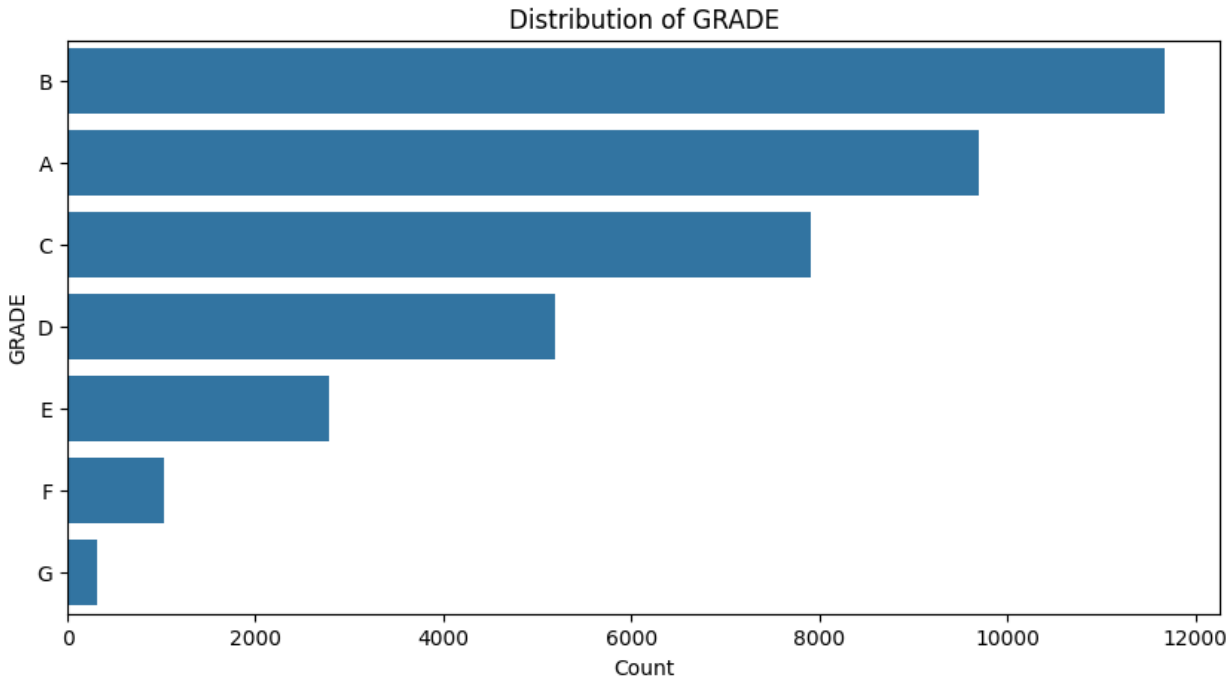
for col in categorical_columns:

    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```




```
categorical_columns = [
    'HOME_OWNERSHIP',
]

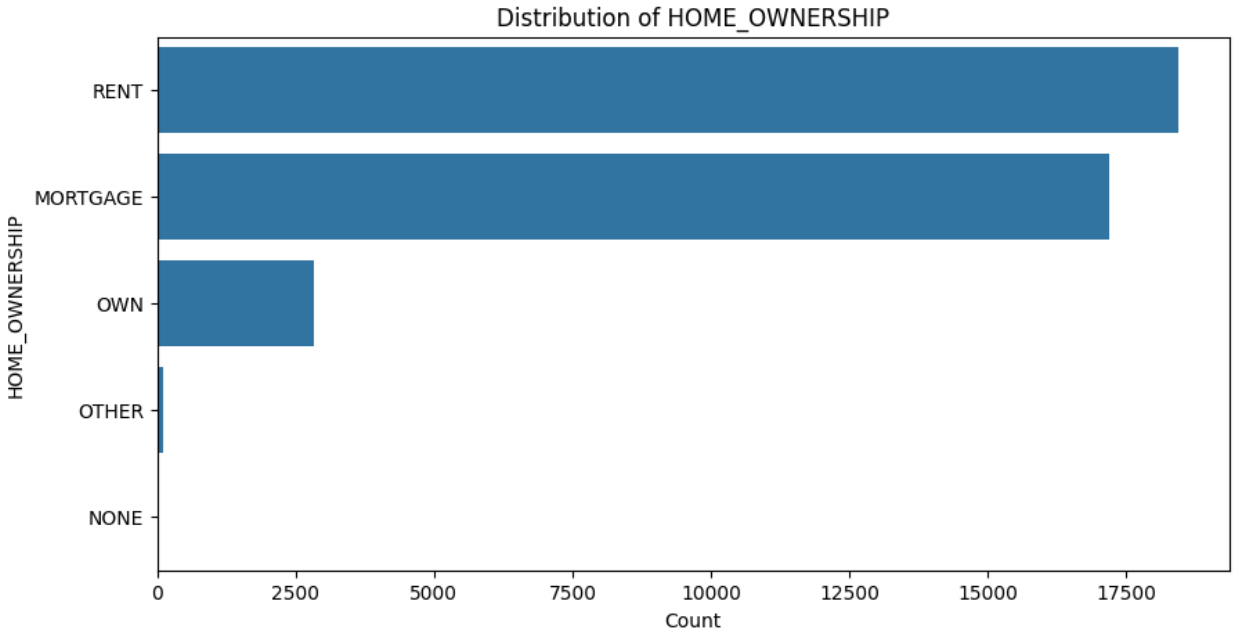
for col in categorical_columns:

    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```




```
categorical_columns = [
    'VERIFICATION_STATUS',
]

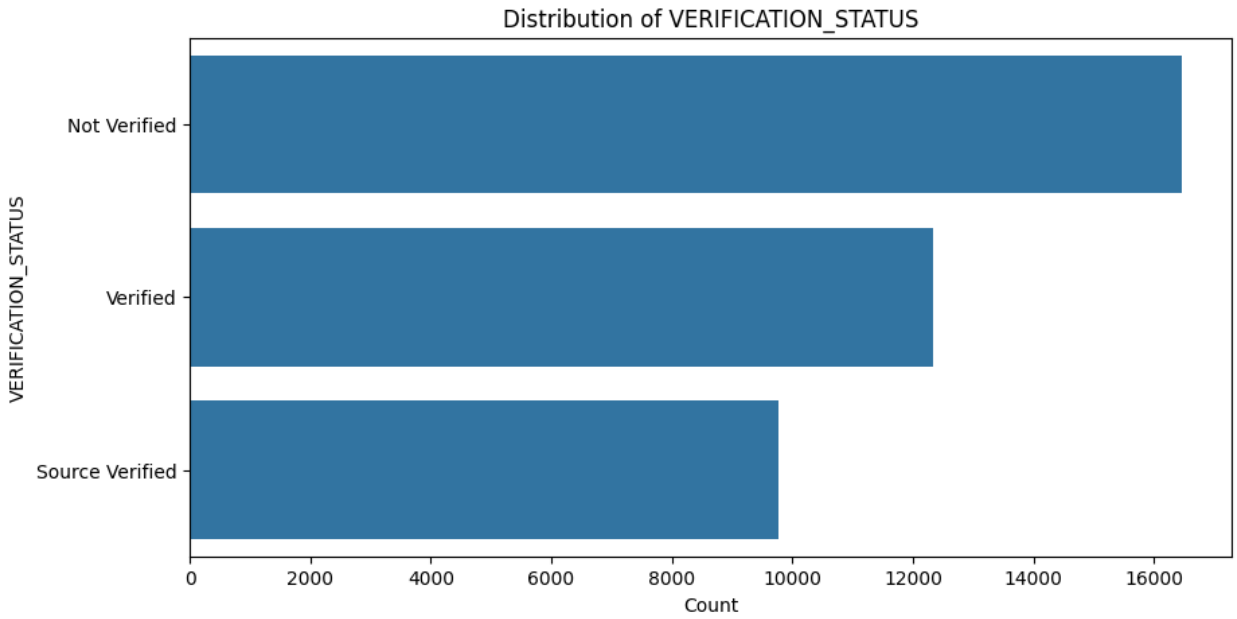
for col in categorical_columns:

    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```




```
categorical_columns = [
    'TERM'
]

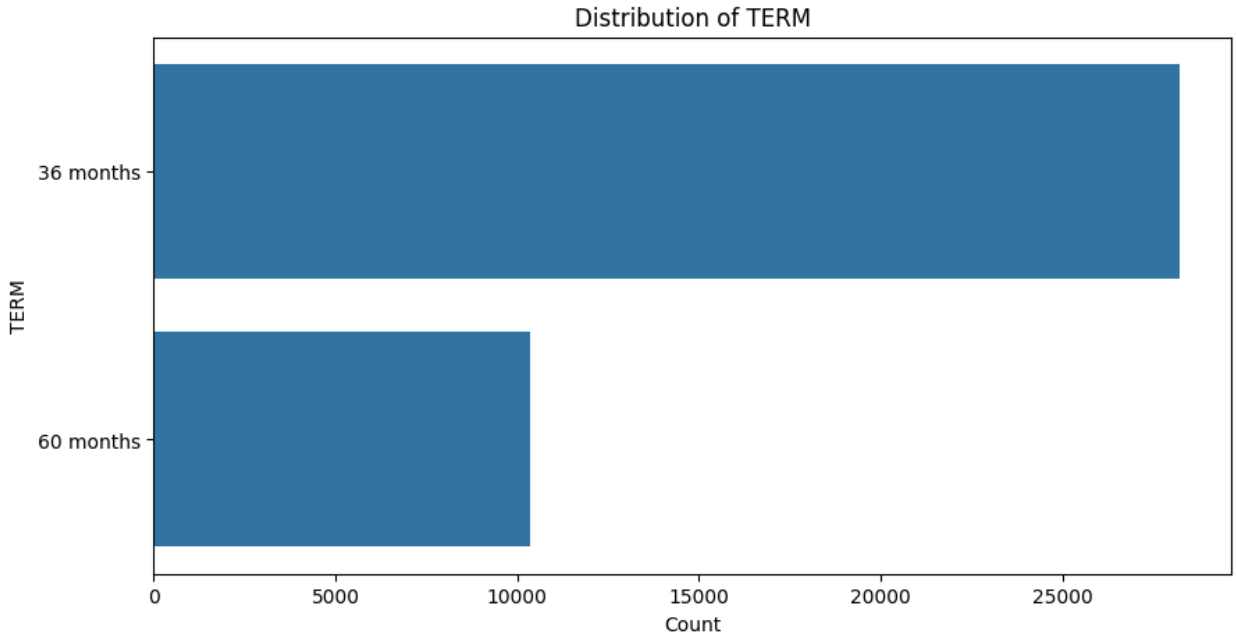
for col in categorical_columns:

    plt.figure(figsize=(10, 5))

    sns.countplot(
        data=df,
        y=col,
        order=df[col].value_counts().index
    )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

    plt.show()
```

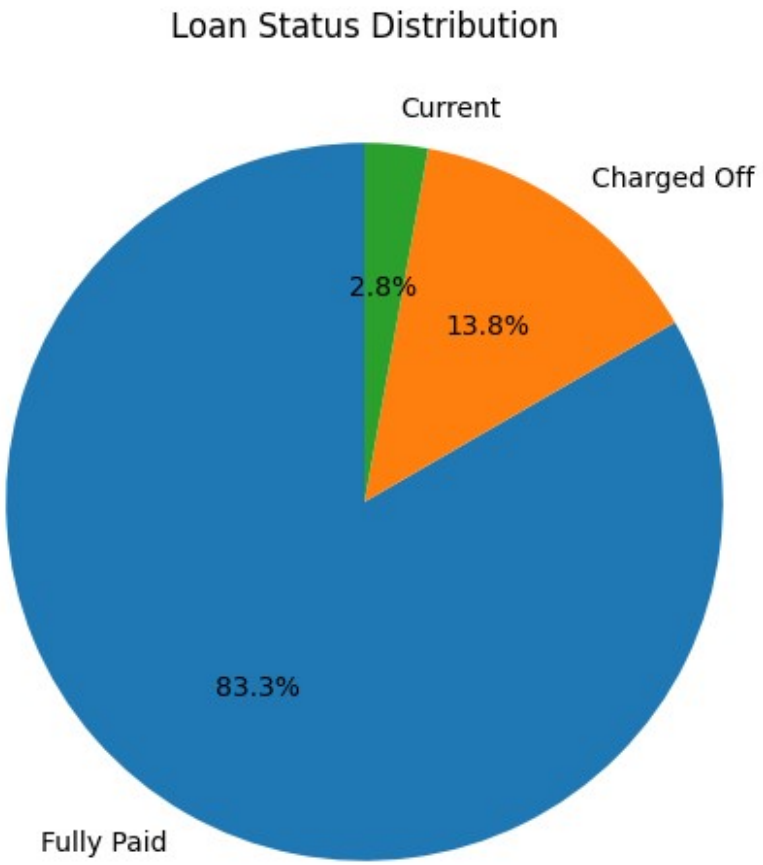



```
loan_status = df['LOAN_STATUS'].value_counts()

plt.figure(figsize=(8, 6))

plt.pie(
    loan_status,
    labels=loan_status.index,
    autopct='%1.1f%%',
    startangle=90
)

plt.title('Loan Status Distribution')
plt.show()
```

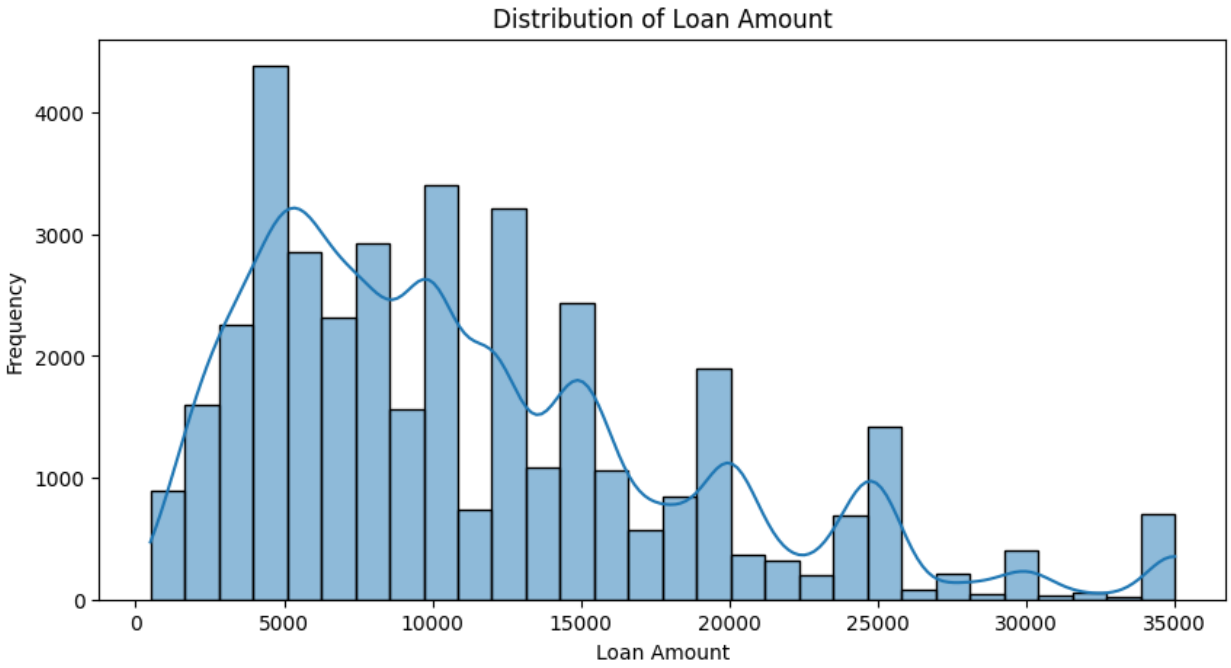



```
plt.figure(figsize=(10, 5))

sns.histplot(df['LOAN_AMOUNT'], bins=30, kde=True)

plt.title('Distribution of Loan Amount')
plt.xlabel('Loan Amount')
plt.ylabel('Frequency')

plt.show()
```



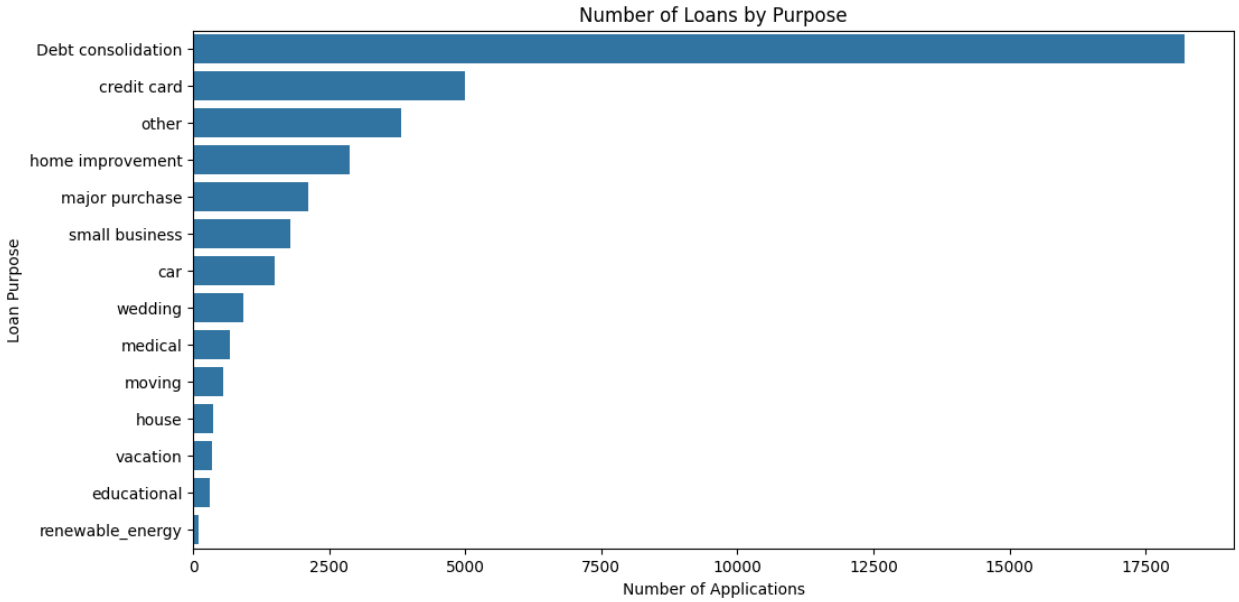

```
plt.figure(figsize=(12, 6))

purpose_data = df['PURPOSE'].value_counts()

sns.barplot(
    x=purpose_data.values,
    y=purpose_data.index
)

plt.title('Number of Loans by Purpose')
plt.xlabel('Number of Applications')
plt.ylabel('Loan Purpose')

plt.show()
```

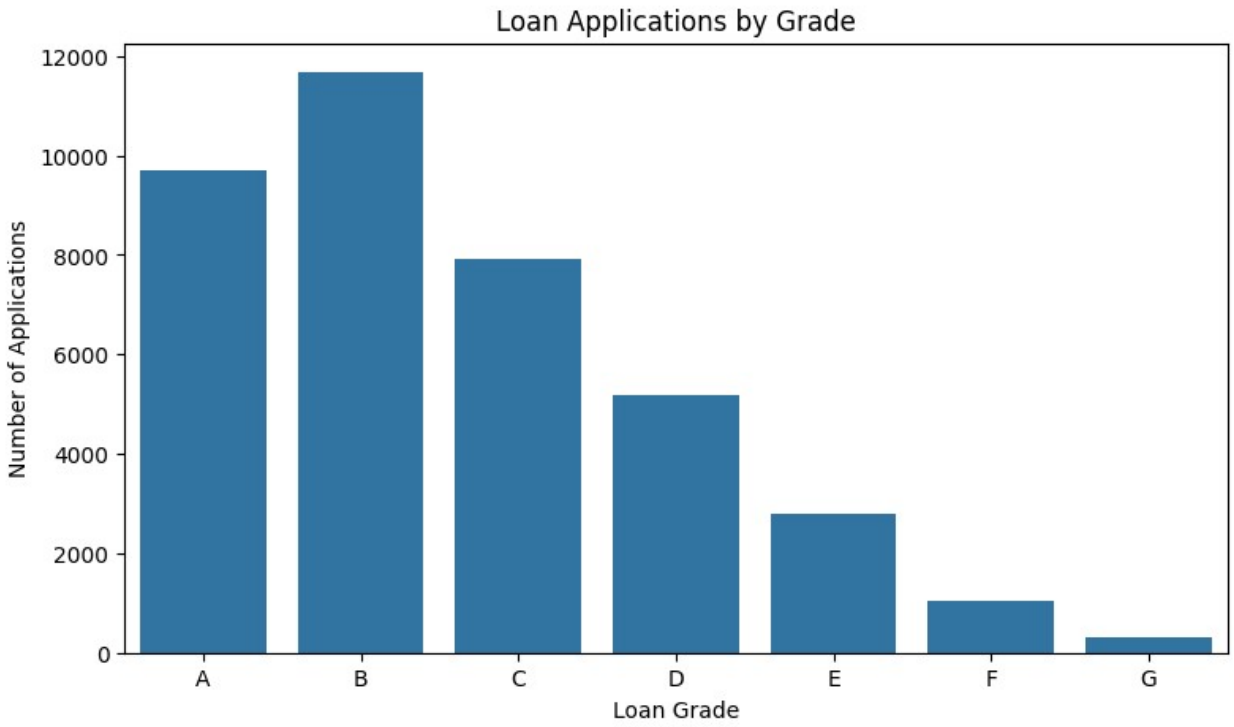



```
plt.figure(figsize=(9, 5))

sns.countplot(
    data=df,
    x='GRADE',
    order=sorted(df['GRADE'].dropna().unique())
)

plt.title('Loan Applications by Grade')
plt.xlabel('Loan Grade')
plt.ylabel('Number of Applications')

plt.show()
```

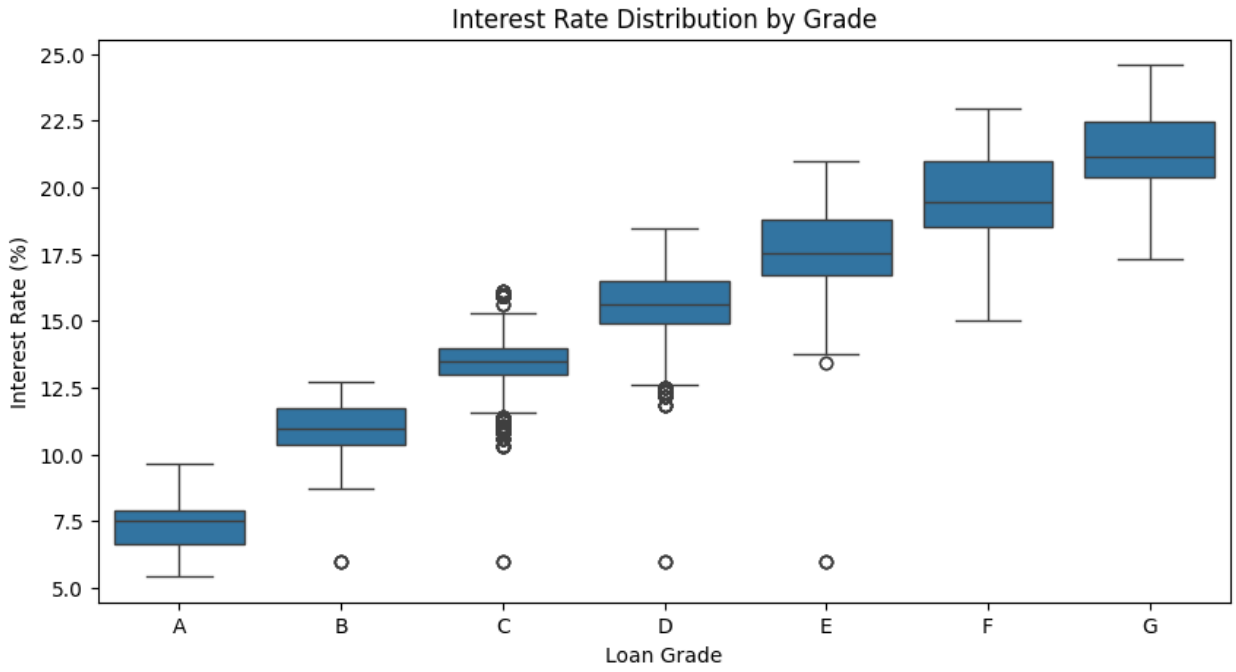



```
plt.figure(figsize=(10, 5))

sns.boxplot(
    data=df,
    x='GRADE',
    y='INTEREST_RATE_PCT',
    order=sorted(df['GRADE'].dropna().unique())
)

plt.title('Interest Rate Distribution by Grade')
plt.xlabel('Loan Grade')
plt.ylabel('Interest Rate (%)')

plt.show()
```

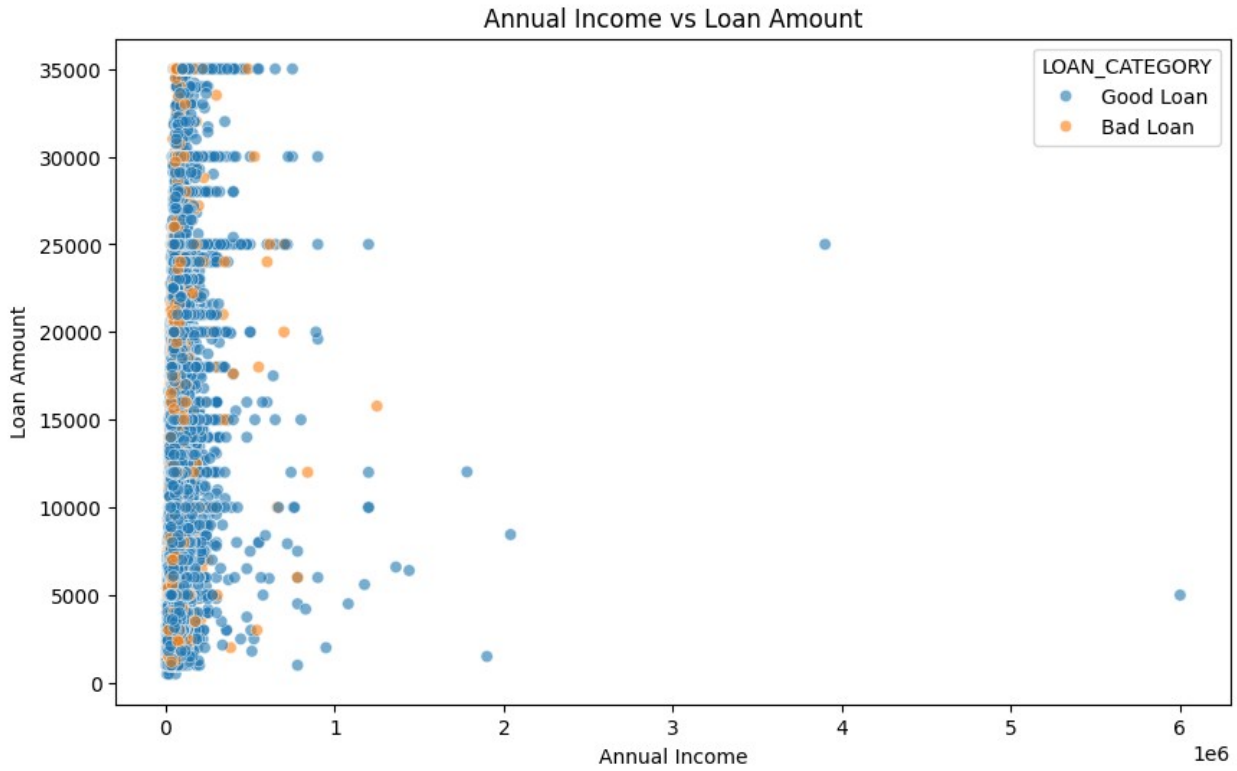



```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='ANNUAL_INCOME',
    y='LOAN_AMOUNT',
    hue='LOAN_CATEGORY',
    alpha=0.6
)

plt.title('Annual Income vs Loan Amount')
plt.xlabel('Annual Income')
plt.ylabel('Loan Amount')

plt.show()
```



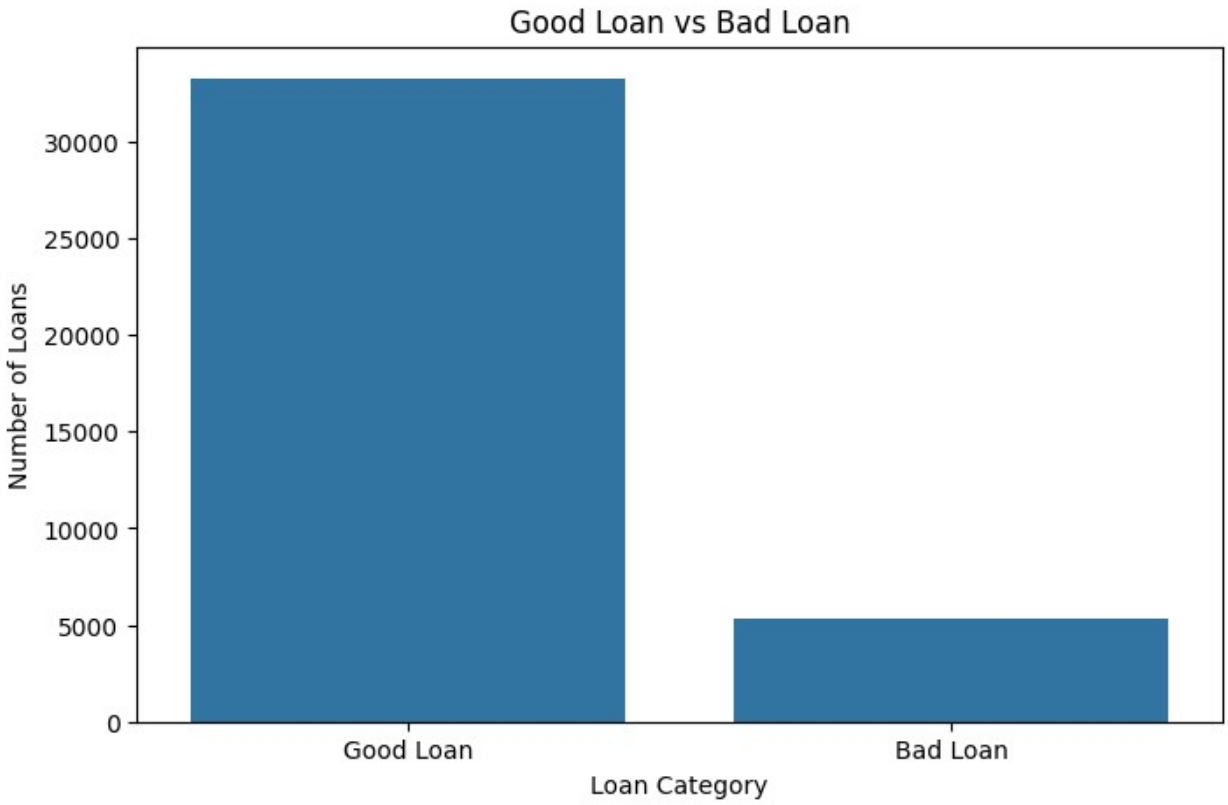

```
loan_category = df['LOAN_CATEGORY'].value_counts()

plt.figure(figsize=(8, 5))

sns.barplot(
    x=loan_category.index,
    y=loan_category.values
)

plt.title('Good Loan vs Bad Loan')
plt.xlabel('Loan Category')
plt.ylabel('Number of Loans')

plt.show()
```

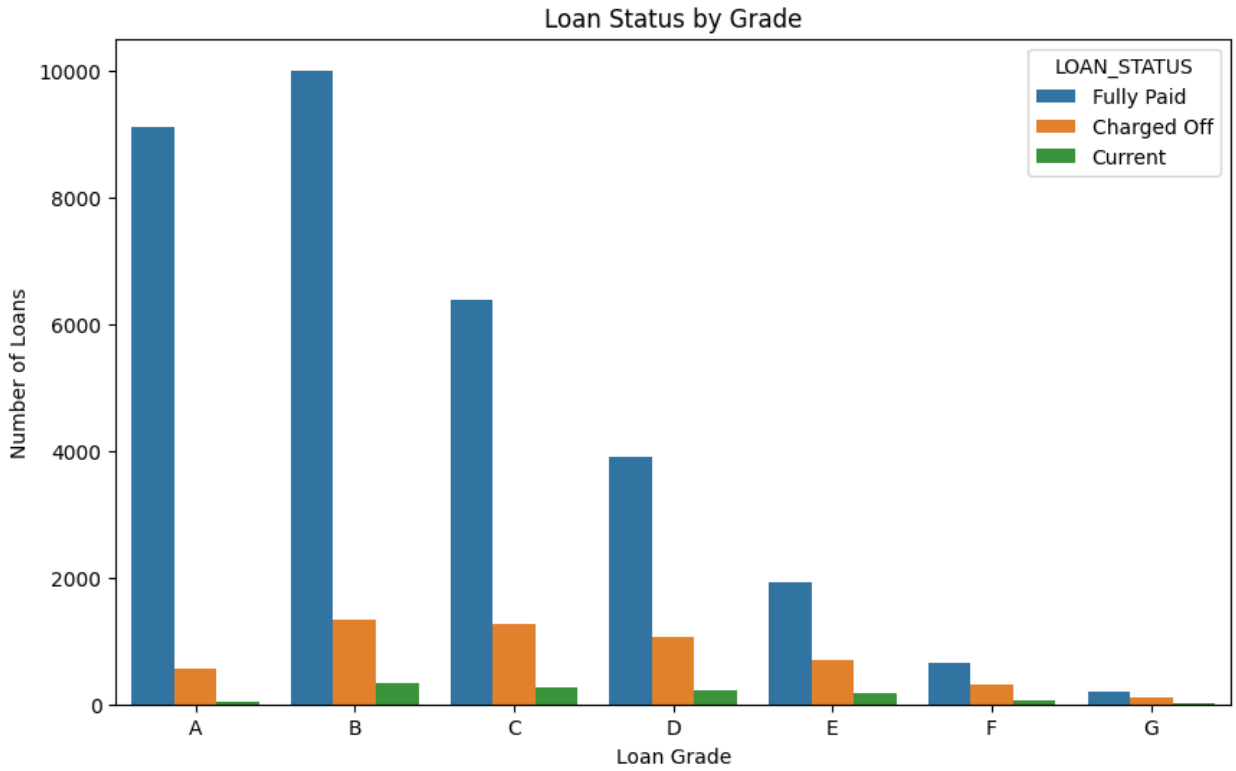



```
plt.figure(figsize=(10, 6))

sns.countplot(
    data=df,
    x='GRADE',
    hue='LOAN_STATUS',
    order=sorted(df['GRADE'].dropna().unique())
)

plt.title('Loan Status by Grade')
plt.xlabel('Loan Grade')
plt.ylabel('Number of Loans')

plt.show()
```

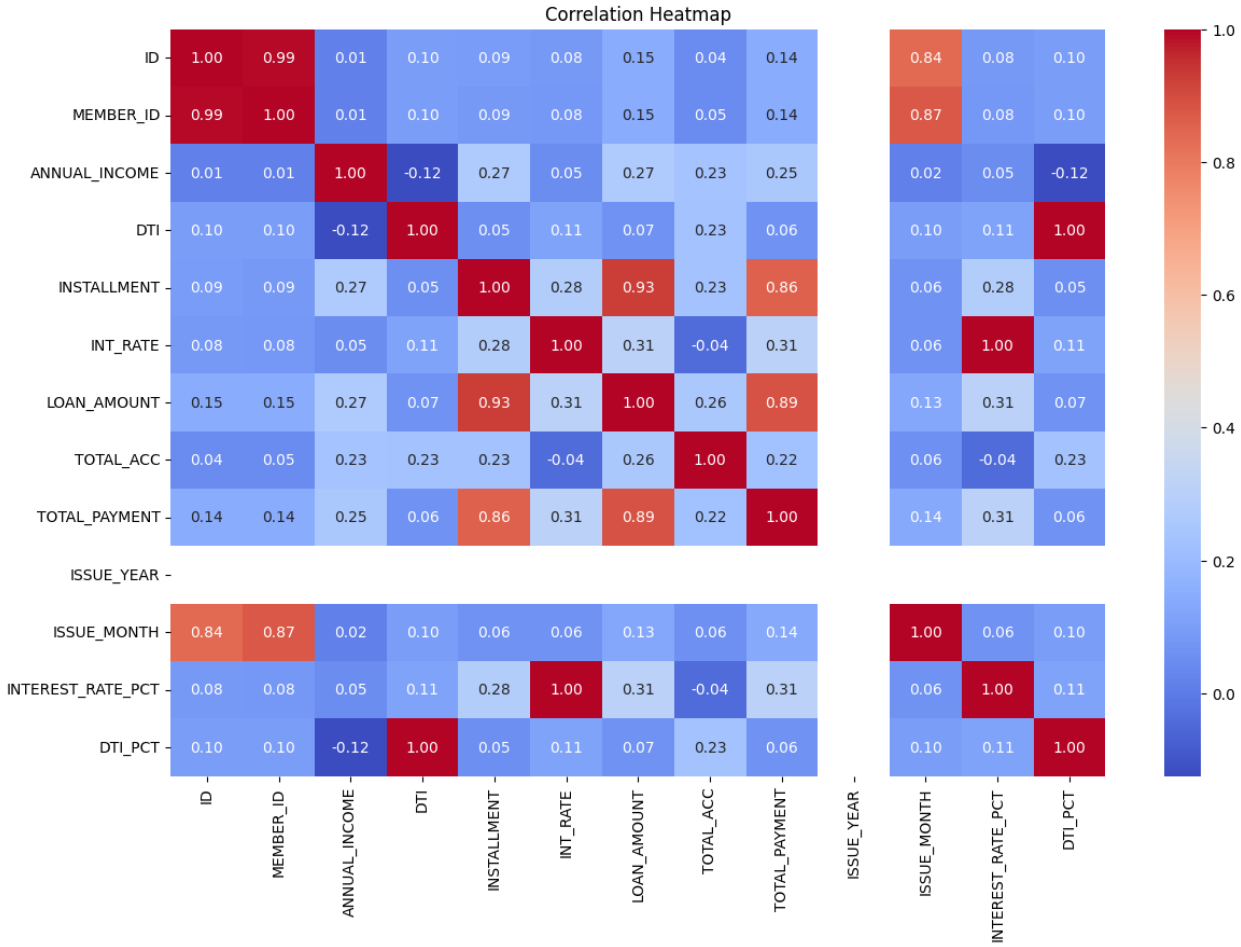



```
correlation = df.select_dtypes(include=np.number).corr()

plt.figure(figsize=(14, 9))

sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Heatmap')
plt.show()
```




```
monthly_loans = df.groupby('ISSUE_MONTH_NAME').size()

month_order = [
    'January', 'February', 'March', 'April',
    'May', 'June', 'July', 'August',
    'September', 'October', 'November', 'December'
]

monthly_loans = monthly_loans.reindex(month_order)

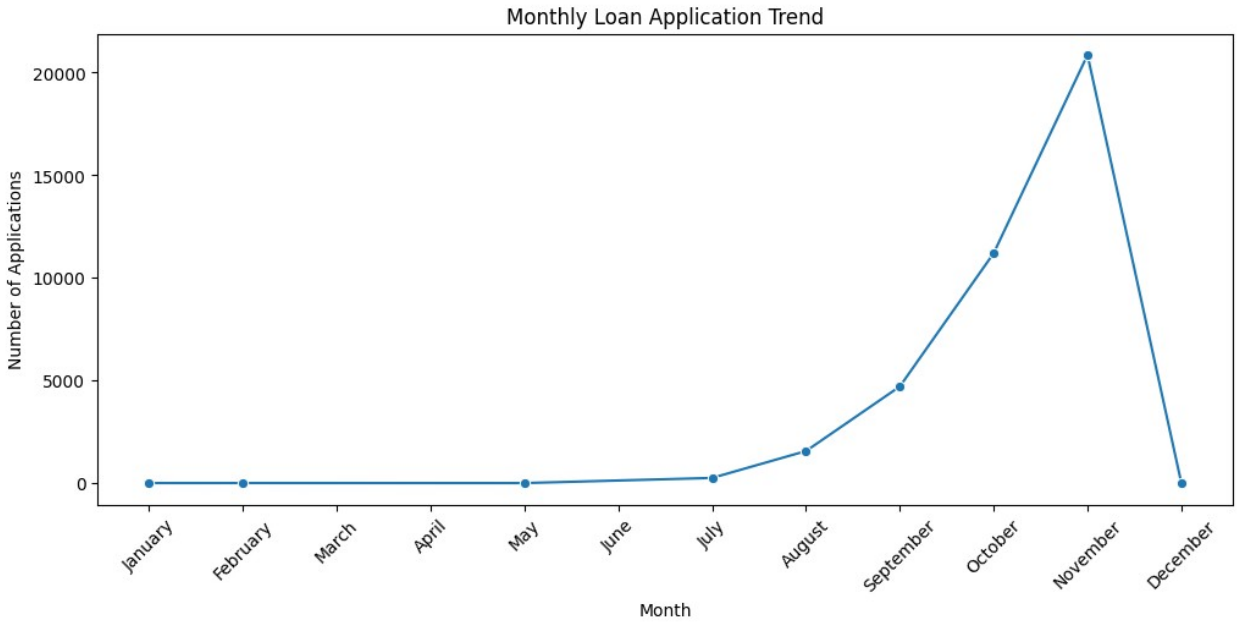
plt.figure(figsize=(12, 5))

sns.lineplot(
    x=monthly_loans.index,
    y=monthly_loans.values,
    marker='o'
)

plt.title('Monthly Loan Application Trend')
plt.xlabel('Month')
plt.ylabel('Number of Applications')
```

```
plt.xticks(rotation=45)

plt.show()
```




```
print("Total Loan Applications:", len(df))

print("Total Funded Amount:", df['LOAN_AMOUNT'].sum())

print("Total Amount Received:", df['TOTAL_PAYMENT'].sum())

print("Average Loan Amount:", df['LOAN_AMOUNT'].mean())

print("Average Interest Rate:", df['INTEREST_RATE_PCT'].mean())

print("Average DTI:", df['DTI_PCT'].mean())

print("\nLoan Status Summary:")
print(df['LOAN_STATUS'].value_counts())

print("\nLoan Category Summary:")
print(df['LOAN_CATEGORY'].value_counts())
```

```
Total Loan Applications: 38576
Total Funded Amount: 435757075
Total Amount Received: 473070933
Average Loan Amount: 11296.066855039402
Average Interest Rate: 12.048831397760265
Average DTI: 13.327433119037744

Loan Status Summary:
```

```
LOAN_STATUS
Fully Paid     32145
Charged Off     5333
Current         1098
Name: count, dtype: Int64

Loan Category Summary:
LOAN_CATEGORY
Good Loan    33243
Bad Loan      5333
Name: count, dtype: int64
```

```
df.to_csv('cleaned_financial_loan_data.csv', index=False)

print("Cleaned dataset exported successfully!")
```

```
Cleaned dataset exported successfully!
```

```
import os

print("Current Folder:", os.getcwd())

print("CSV Exists:",
os.path.exists('cleaned_financial_loan_data.csv'))
```

```
Current Folder: C:\Users\hamsa\Desktop\Jupyter projects
CSV Exists: True
```

```
# Standardize column names
df.columns = df.columns.str.strip().str.upper()

# Expected original PostgreSQL columns
original_columns = [
    'ID',
    'ADDRESS_STATE',
    'APPLICATION_TYPE',
    'EMP_LENGTH',
    'EMP_TITLE',
    'GRADE',
    'HOME_OWNERSHIP',
    'ISSUE_DATE',
    'LAST_CREDIT_PULL_DATE',
    'LAST_PAYMENT_DATE',
    'LOAN_AMOUNT',
    'LOAN_STATUS',
    'PURPOSE',
    'SUB_GRADE',
    'TERM',
    'VERIFICATION_STATUS',
    'ANNUAL_INCOME',
    'DTI',
```

```
    'INSTALLMENT',
    'INT_RATE',
    'TOTAL_ACC',
    'TOTAL_PAYMENT',
    'NEXT_PAYMENT_DATE'
]

# Identify missing columns
missing_columns = [
    col for col in original_columns if col not in df.columns
]

print("Missing columns:", missing_columns)

# Add missing columns with empty values
for col in missing_columns:
    df[col] = None

# Select the required columns
postgres_df = df[original_columns].copy()

# Convert column names to lowercase
postgres_df.columns = postgres_df.columns.str.lower()

# Export CSV
postgres_df.to_csv(
    'financial_loan_postgres.csv',
    index=False,
    encoding='utf-8'
)

print("\nCSV exported successfully!")
print("Dataset shape:", postgres_df.shape)

postgres_df.head()
```

```
Missing columns: ['APPLICATION_TYPE']

CSV exported successfully!
Dataset shape: (38576, 23)
```

	id	address_state	application_type	emp_length	emp_title
0	1072053	CA	None	9 years	MKC Accounting
1	1068350	IL	None	10+ years	J&J Steel Inc
2	1069243	CA	None	4 years	Chemat Technology Inc
3	207910	FL	None	< 1 year	Unknown

```
4   211723            MD             None    4 years           self
employed

  grade home_ownership issue_date last_credit_pull_date
last_payment_date  \
0     E           RENT 2021-01-01            2021-12-14         2021-
01-15
1     A       MORTGAGE 2021-01-01            2021-12-14         2021-
01-15
2     C           RENT 2021-05-01            2021-12-12         2021-
01-09
3     A       MORTGAGE 2021-08-01            2021-05-16         2021-
02-10
4     C            OWN 2021-08-01            2021-12-09         2021-
08-08

   ...  sub_grade       term verification_status annual_income     dti
\
0  ...         E1  36 months     Source Verified       48000.0  0.0535

1  ...         A1  36 months            Verified       83000.0  0.0231

2  ...         C5  36 months        Not Verified       50000.0  0.2088

3  ...         A2  36 months        Not Verified      120000.0  0.0767

4  ...         C2  36 months        Not Verified       72000.0  0.0512


  installment  int_rate  total_acc  total_payment  next_payment_date
0      109.43    0.1864          4           3939         2021-02-15
1      106.53    0.0603         28           3835         2021-02-15
2      421.65    0.1596         11           3522         2021-02-09
3       69.14    0.0743         20           2508         2021-03-10
4      262.26    0.1109          8           2135         2021-09-08

[5 rows x 23 columns]

check_df = pd.read_csv('financial_loan_postgres.csv')

print("Rows and Columns:", check_df.shape)

print("\nColumn Names:")
print(check_df.columns.tolist())

check_df.head()

Rows and Columns: (38576, 23)

Column Names:
['id', 'address_state', 'application_type', 'emp_length', 'emp_title',
```

```
'grade', 'home_ownership', 'issue_date', 'last_credit_pull_date',
'last_payment_date', 'loan_amount', 'loan_status', 'purpose',
'sub_grade', 'term', 'verification_status', 'annual_income', 'dti',
'installment', 'int_rate', 'total_acc', 'total_payment',
'next_payment_date']

        id address_state  application_type emp_length
emp_title  \
0  1072053            CA               NaN    9 years          MKC
Accounting
1  1068350            IL               NaN  10+ years           J&J
Steel Inc
2  1069243            CA               NaN    4 years  Chemat
Technology Inc
3   207910            FL               NaN   < 1 year
Unknown
4   211723            MD               NaN    4 years           self
employed

  grade home_ownership  issue_date last_credit_pull_date
last_payment_date  \
0     E           RENT  2021-01-01            2021-12-14         2021-
01-15
1     A       MORTGAGE  2021-01-01            2021-12-14         2021-
01-15
2     C           RENT  2021-05-01            2021-12-12         2021-
01-09
3     A       MORTGAGE  2021-08-01            2021-05-16         2021-
02-10
4     C            OWN  2021-08-01            2021-12-09         2021-
08-08

   ...  sub_grade       term verification_status annual_income     dti
\
0  ...         E1  36 months     Source Verified       48000.0  0.0535

1  ...         A1  36 months            Verified       83000.0  0.0231

2  ...         C5  36 months        Not Verified       50000.0  0.2088

3  ...         A2  36 months        Not Verified      120000.0  0.0767

4  ...         C2  36 months        Not Verified       72000.0  0.0512


  installment  int_rate  total_acc  total_payment  next_payment_date
0      109.43    0.1864          4           3939         2021-02-15
1      106.53    0.0603         28           3835         2021-02-15
2      421.65    0.1596         11           3522         2021-02-09
3       69.14    0.0743         20           2508         2021-03-10
```

```
4      262.26    0.1109          8           2135         2021-09-08

[5 rows x 23 columns]
```

```
import pandas as pd

# Load the current CSV
df = pd.read_csv('financial_loan_postgres.csv')

# Check application_type missing values
print("Missing values:")
print(df['application_type'].isnull().sum())

# Remove application_type column
df.drop(columns=['application_type'], inplace=True)

# Export corrected CSV
df.to_csv(
    'financial_loan_postgres.csv',
    index=False,
    encoding='utf-8'
)

print("\nColumn removed successfully!")
print("New Dataset Shape:", df.shape)

display(df.head())
```

```
Missing values:
38576

Column removed successfully!
New Dataset Shape: (38576, 22)

        id address_state emp_length              emp_title grade  \
0  1072053            CA    9 years         MKC Accounting     E
1  1068350            IL  10+ years          J&J Steel Inc     A
2  1069243            CA    4 years  Chemat Technology Inc     C
3   207910            FL   < 1 year                Unknown     A
4   211723            MD    4 years           self employed     C

  home_ownership  issue_date last_credit_pull_date
last_payment_date  \
0           RENT  2021-01-01            2021-12-14        2021-01-15

1       MORTGAGE  2021-01-01            2021-12-14        2021-01-15

2           RENT  2021-05-01            2021-12-12        2021-01-09

3       MORTGAGE  2021-08-01            2021-05-16        2021-02-10
```

```
4            OWN  2021-08-01            2021-12-09        2021-08-08


   loan_amount  ... sub_grade       term verification_status
annual_income  \
0         3000  ...        E1  36 months     Source Verified
48000.0
1         3500  ...        A1  36 months            Verified
83000.0
2        12000  ...        C5  36 months        Not Verified
50000.0
3         2225  ...        A2  36 months        Not Verified
120000.0
4         8000  ...        C2  36 months        Not Verified
72000.0

      dti  installment  int_rate  total_acc  total_payment
next_payment_date
0  0.0535       109.43    0.1864          4           3939
2021-02-15
1  0.0231       106.53    0.0603         28           3835
2021-02-15
2  0.2088       421.65    0.1596         11           3522
2021-02-09
3  0.0767        69.14    0.0743         20           2508
2021-03-10
4  0.0512       262.26    0.1109          8           2135
2021-09-08

[5 rows x 22 columns]

df.columns

Index(['id', 'address_state', 'emp_length', 'emp_title', 'grade',
       'home_ownership', 'issue_date', 'last_credit_pull_date',
       'last_payment_date', 'loan_amount', 'loan_status', 'purpose',
       'sub_grade', 'term', 'verification_status', 'annual_income',
'dti',
       'installment', 'int_rate', 'total_acc', 'total_payment',
       'next_payment_date'],
      dtype='object')

# Export CSV
postgres_df.to_csv(
    'financial_loan_postgres.csv',
    index=False,
    encoding='utf-8'
)
```

```
from IPython.display import FileLink, display

display(FileLink('financial_loan_postgres.csv'))
```

```
C:\Users\hamsa\Desktop\Jupyter projects\financial_loan_postgres.csv
```

```
# Export CSV
postgres_df.to_csv(
    'financial_loan_postgres.csv',
    index=False,
    encoding='utf-8'
)

print("\nCSV exported successfully!")
print("Dataset shape:", postgres_df.shape)

postgres_df.head()
```

```
CSV exported successfully!
Dataset shape: (38576, 23)

        id address_state application_type emp_length
emp_title  \
0  1072053            CA             None    9 years          MKC
Accounting
1  1068350            IL             None  10+ years           J&J
Steel Inc
2  1069243            CA             None    4 years  Chemat
Technology Inc
3   207910            FL             None   < 1 year
Unknown
4   211723            MD             None    4 years           self
employed

  grade home_ownership issue_date last_credit_pull_date
last_payment_date  \
0     E           RENT 2021-01-01            2021-12-14        2021-
01-15
1     A       MORTGAGE 2021-01-01            2021-12-14        2021-
01-15
2     C           RENT 2021-05-01            2021-12-12        2021-
01-09
3     A       MORTGAGE 2021-08-01            2021-05-16        2021-
02-10
4     C            OWN 2021-08-01            2021-12-09        2021-
08-08

   ...  sub_grade       term verification_status annual_income     dti
\
0  ...         E1  36 months     Source Verified       48000.0  0.0535
```

```
1  ...         A1  36 months            Verified       83000.0  0.0231

2  ...         C5  36 months        Not Verified       50000.0  0.2088

3  ...         A2  36 months        Not Verified      120000.0  0.0767

4  ...         C2  36 months        Not Verified       72000.0  0.0512


  installment  int_rate  total_acc  total_payment  next_payment_date
0      109.43     0.1864          4           3939         2021-02-15
1      106.53     0.0603         28           3835         2021-02-15
2      421.65     0.1596         11           3522         2021-02-09
3       69.14     0.0743         20           2508         2021-03-10
4      262.26     0.1109          8           2135         2021-09-08

[5 rows x 23 columns]
```

```
import os

print("Current Folder:", os.getcwd())

print("CSV Exists:", os.path.exists('financial_loan_postgres.csv'))
```

```
Current Folder: C:\Users\hamsa\Desktop\Jupyter projects
CSV Exists: True
```

```
import pandas as pd

# Load dataset
df = pd.read_csv('financial_loan_postgres.csv')

# Remove empty application_type column
if 'application_type' in df.columns:
    df.drop(columns=['application_type'], inplace=True)

# Save cleaned dataset
df.to_csv('financial_loan_postgres.csv', index=False, encoding='utf-
8')

print("CSV file saved successfully!")

# Verify dataset
print("Dataset Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

```
CSV file saved successfully!
Dataset Shape: (38576, 22)
Columns: ['id', 'address_state', 'emp_length', 'emp_title', 'grade',
'home_ownership', 'issue_date', 'last_credit_pull_date',
```

```
'last_payment_date', 'loan_amount', 'loan_status', 'purpose',
'sub_grade', 'term', 'verification_status', 'annual_income', 'dti',
'installment', 'int_rate', 'total_acc', 'total_payment',
'next_payment_date']
```